



Lucrare de Laborator 1

Procesarea Semnalelor

Elaborat: Macrii Danu
Gr.: FAF-222

Chişinău, March 2025

Contents

Cercetarea proceselor aleatorii	3
1.1 Zgomotul „alb” cu reprezentare după dependența gauss	3
1.2 Reprezențați histograma zgomotului generat	3
1.3 Repetați p. 1.1 pentru $T_s=0.001$	4
1.4 Reprezențați histograma zgomotului generat x2	4
1.5 Proiectați un filtru digital de ordinul doi	5
1.6 Repetați p. 1.5 pentru $T_s=0.001$	6
Filtrarea semnalelor afectate de zgomot folosind un filtru MAF	6
2.1 Generați un semnal original neafectat de zgomot	6
2.2 Generați un zgomot	7
2.3 Reprezențați semnalele pe un grafic continuu	8
2.4 Reprezențați suma semnalelor	8
2.5 Proiectați un filtru MAF	8
2.6 Repetați p. 2.5 pentru $M=5$ și $M=10$	9
2.7 Repetați p. 2.5 pentru un alt semnal	10
2.8 Repetați p. 2.7 pentru $M=50$ și $M=100$	10
Concluzii	12

Cercetarea proceselor aleatorii

1.1 Zgomotul „alb” cu reprezentare după dependența gauss

Generați un zgomot alb

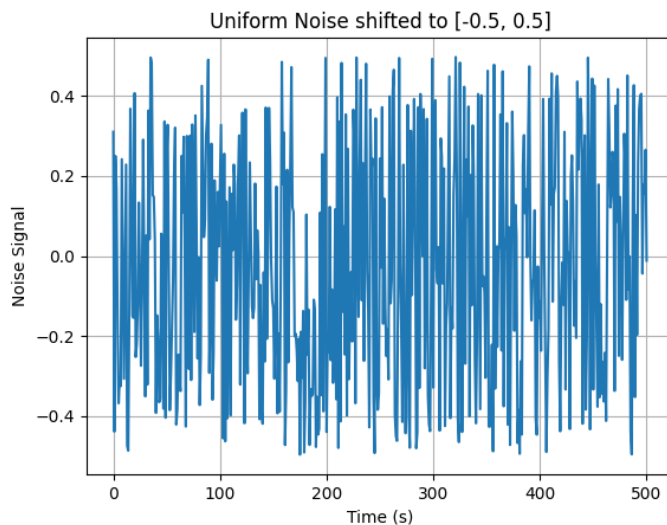
main.cpp

```
lower_limit = 0
upper_limit = 5
step = 0.01
size = int((upper_limit - lower_limit) / step) + 1 # Include last value

samples = np.random.rand(size) - 0.5 # Shift uniform distribution to [-0.5, 0.5]

t = np.linspace(lower_limit, upper_limit, size) # Create time vector

plt.plot(t, samples)
```



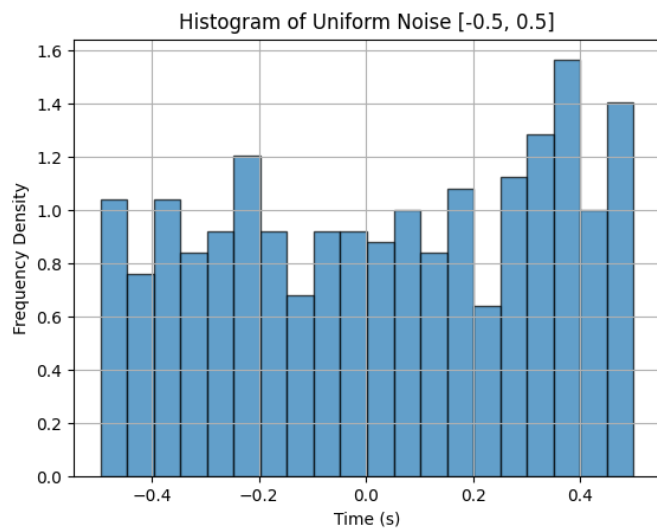
1.2 Reprezentați histograma zgomotului generat

Înlocuiți funcția ‘plot’ cu funcția ‘hist’ pentru a reprezenta histograma zgomotului generat. Schimbați timpul până la 1.

```
lower_limit = 0
upper_limit = 5
step = 0.001
t = np.arange(lower_limit, upper_limit, step)

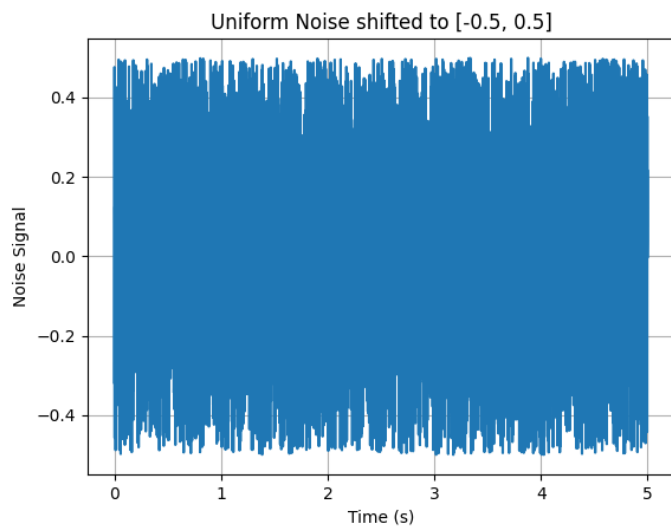
samples = np.random.rand(len(t)) - 0.5

plt.hist(samples, bins=20, edgecolor='black', alpha=0.7, density=True)
```



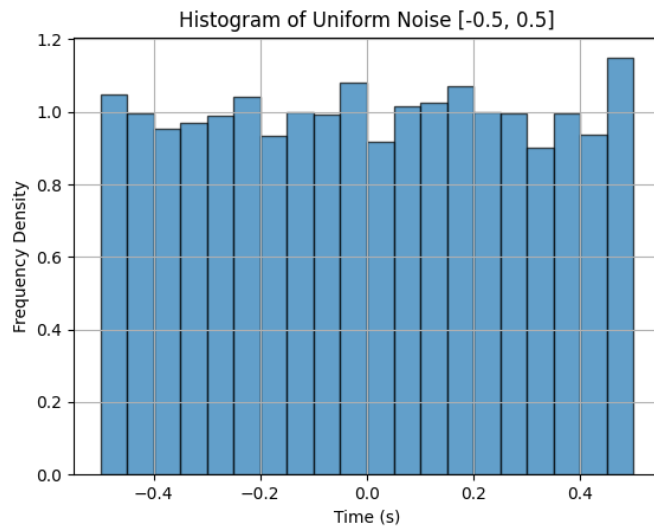
1.3 Repetați p. 1.1 pentru $T_s=0.001$

Generați un nou zgomot x_2 cu pasul de timp $T_s = 0.001$:



1.4 Reprezentați histograma zgomotului generat x_2

Înlocuiți funcția 'plot' cu funcția 'hist' pentru a reprezenta histograma zgomotului generat x_2 .



1.5 Proiectați un filtru digital de ordinul doi

Proiectați un filtru digital de ordinul doi cu frecvența oscilațiilor proprii de 1 Hz și lansați semnalul x_1 prin acest filtru:

```
Ts = 0.001
om0 = 2 * np.pi
dz = 0.005
A = 1
oms = om0 * Ts

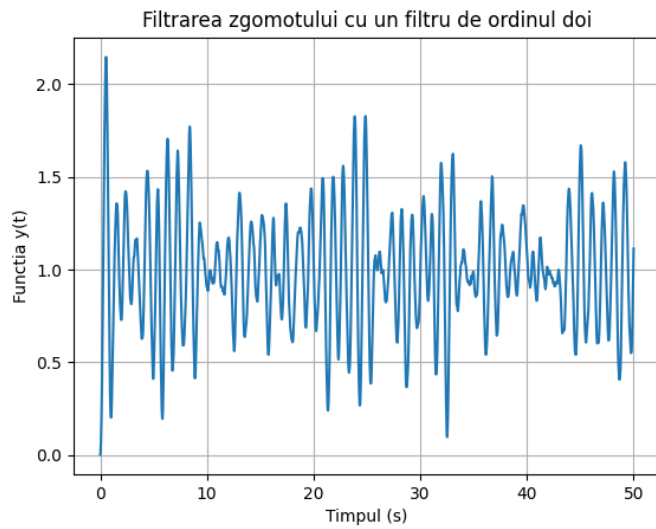
a = [1 + 2 * dz * oms + oms**2, -2 * (1 + dz * oms), 1]
b = [A * 2 * oms**2]

t = np.arange(0, 50 + Ts, Ts)

x1 = np.random.rand(len(t))

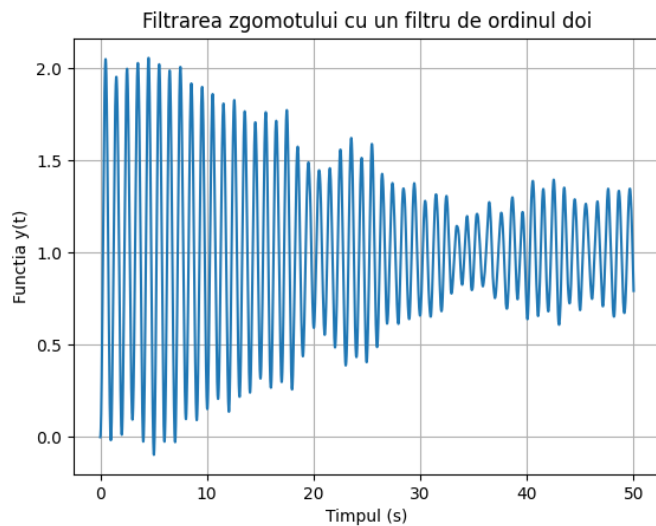
y1 = lfilter(b, a, x1)

plt.plot(t, y1)
```



1.6 Repetați p. 1.5 pentru $T_s=0.001$

Repetăți filtrarea semnalului pentru $T_s = 0.001$ și zgomotul generat x_2 .



Filtrarea semnalelor afectate de zgomot folosind un filtru MAF

2.1 Generați un semnal original neafectat de zgomot

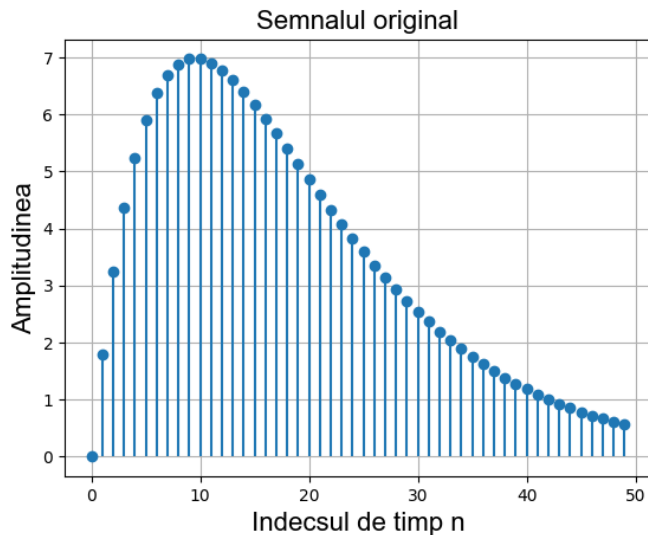
Generați semnalul original $s(n)$:

```
R = 50
m = np.arange(0, R)
s = 2 * m * (0.9 ** m)

plt.stem(m, s, basefmt=" ")
plt.grid(True)
```

```
plt.xlabel('Indecsul de timp n', fontsize=16, family='Arial')
plt.ylabel('Amplitudinea', fontsize=16, family='Arial')
plt.title('Semnalul original', fontsize=16, family='Arial')

plt.show()
```



2.2 Generați un zgomot

Adăugați zgomotul generat la semnalul s :

```
R = 50
m = np.arange(0, R)
s = 2 * m * (0.9 ** m)

d = np.random.rand(len(m)) - 0.5

s_noisy = s + d

plt.plot(m, s, label='Semnalul original')
plt.plot(m, s_noisy, label='Semnalul cu zgomot')

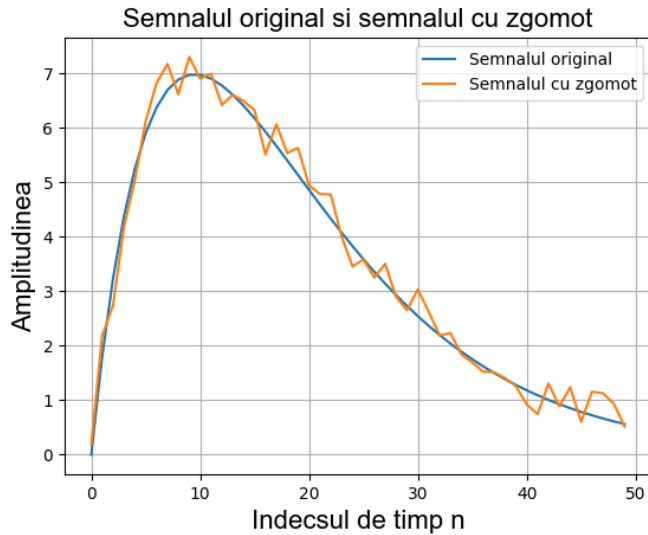
plt.xlabel('Indecsul de timp n', fontsize=16, family='Arial')
plt.ylabel('Amplitudinea', fontsize=16, family='Arial')
plt.title('Semnalul original si semnalul cu zgomot', fontsize=16, family='Arial')

plt.grid(True)
plt.legend()

plt.show()
```

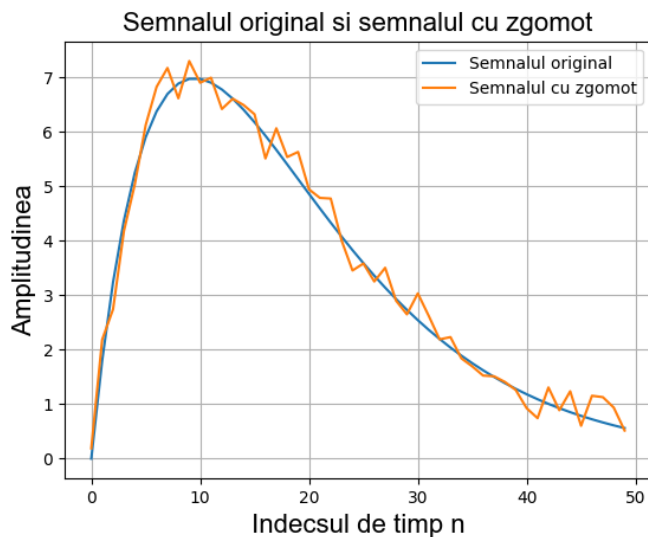
2.3 Reprezentați semnalele pe un grafic continuu

Reprezentați semnalele s și zgomotul d pe același grafic folosind funcția 'plot'.



2.4 Reprezentați suma semnalelor

Reprezentați suma semnalelor $x = s + d$ și compară semnalul inițial s cu semnalul rezultat x pe același grafic.



2.5 Proiectați un filtru MAF

Proiectați un filtru MAF cu parametrii $y = \text{filter}(b, 1, x)$ unde $b = \text{ones}(M, 1)/M$ și $M = 3$. Filtrați semnalul x afectat de zgomot și reprezentați semnalele y , x , și s pe același grafic.

```
# Parameters
```

```
R = 50
```



```

M = 100
m = np.arange(0, R, 0.001)
# s = 2 * m * (0.9 ** m)

s = 2 * sawtooth( 3*np.pi*m +np.pi/6)

d = np.random.rand(len(m)) - 0.5

x = s + d

b = np.ones(M) / M

y = np.convolve(x, b, mode='same')

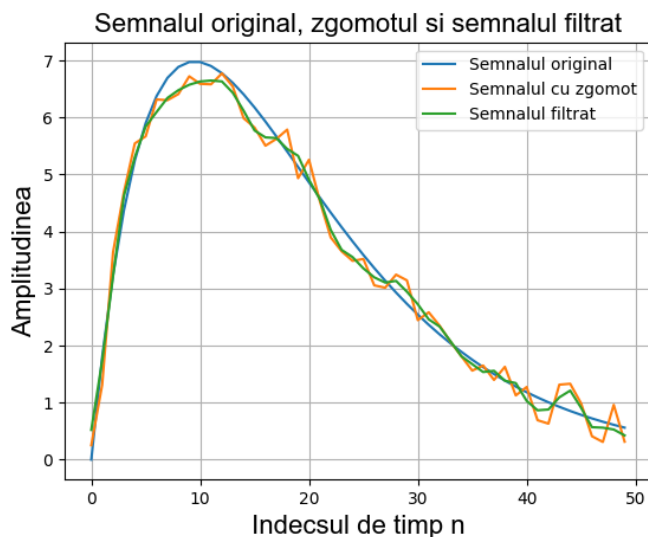
plt.plot(m, s, label='Semnalul original')
plt.plot(m, x, label='Semnalul cu zgomot')
plt.plot(m, y, label='Semnalul filtrat')

plt.xlabel('Indecsul de timp n', fontsize=16, family='Arial')
plt.ylabel('Amplitudinea', fontsize=16, family='Arial')
plt.title('Semnalul original, zgomotul si semnalul filtrat', fontsize=16, family='Arial')

plt.grid(True)
plt.legend()

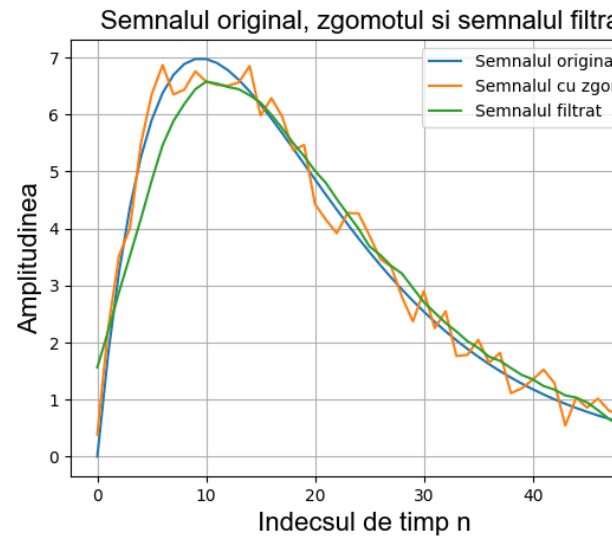
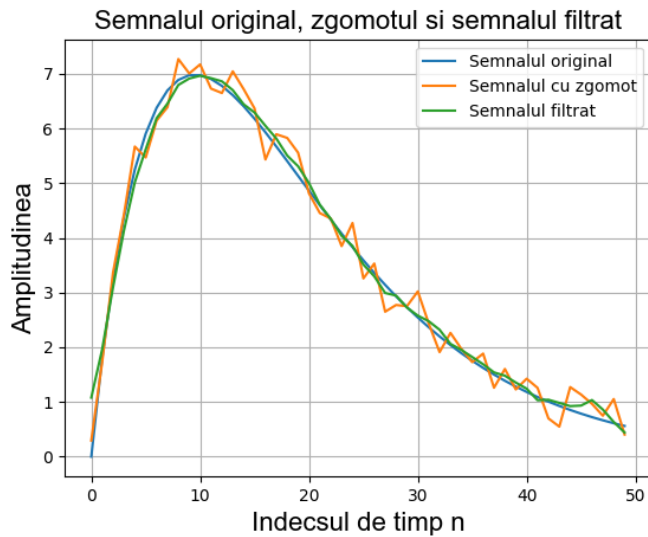
plt.show()

```



2.6 Repetați p. 2.5 pentru $M=5$ și $M=10$

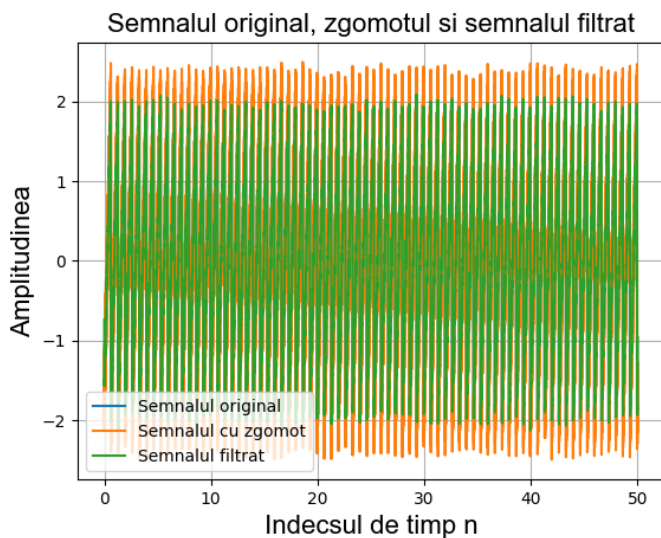
Repetăți filtrarea semnalului cu valorile $M = 5$ și $M = 10$ și comparați rezultatele obținute.



2.7 Repetați p. 2.5 pentru un alt semnal

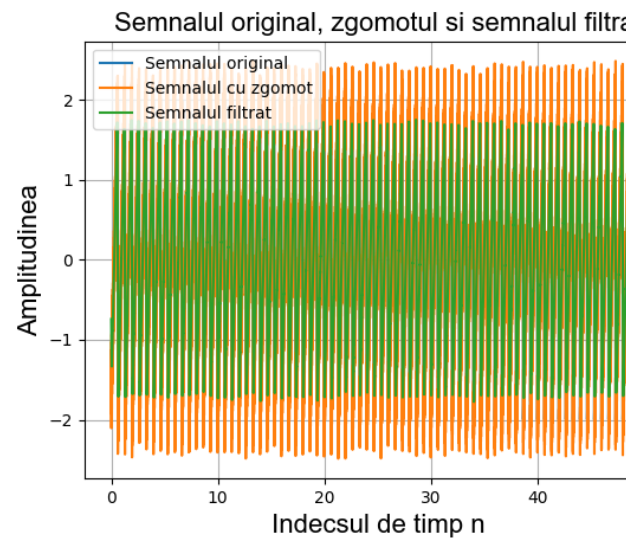
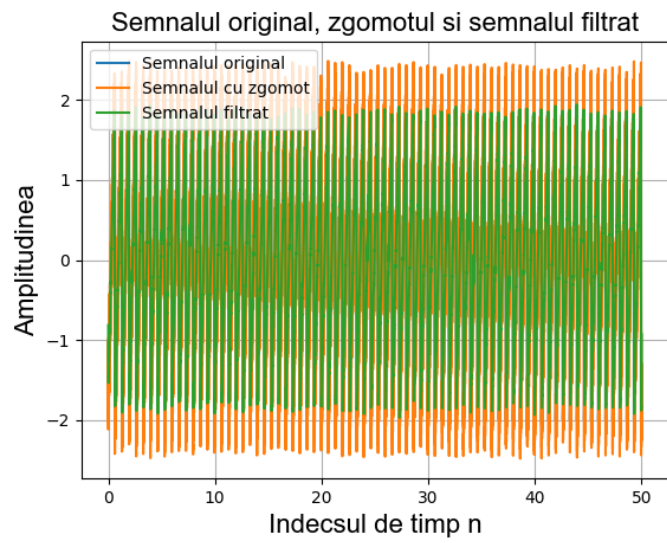
Repetăți filtrarea pentru un semnal diferit, $s = 2 \cdot \text{sawtooth}(3\pi m + \pi/6)$ cu pasul de timp $m = 0 : 0.001 : R$, unde $R = 1$ și $M = 20$. Afișați semnalele filtrate, afectate de zgomot și inițial pe același grafic.

```
s = 2 * sawtooth( 3*np.pi*m +np.pi/6)
```



2.8 Repetați p. 2.7 pentru M=50 și M=100

Repetăți filtrarea pentru valorile $M = 50$ și $M = 100$ și comparați rezultatele obținute.



Concluzii

În cadrul cercetării proceselor aleatorii, s-a observat că zgomotul alb generat folosind funcția **rand** prezintă o distribuție uniformă, iar modificarea timpului de eșantionare influențează frecvența de generare a semnalului, observându-se o detaliere mai mare a semnalului pentru valori mai mici ale lui **Ts**. Histogramele generate pentru zgomotul alb au confirmat această distribuție uniformă.

Filtrarea semnalului cu un filtru digital de ordinul doi a demonstrat eficiența acestuia în atenuarea anumitor frecvențe din zgomotul alb, iar ajustările parametrilor filtrului au avut un impact direct asupra rezultatelor obținute. Compararea semnalului filtrat cu semnalul inițial a oferit o viziune clară asupra efectului filtrării asupra semnalului de intrare.

În cadrul procesului de filtrare a semnalelor afectate de zgomot utilizând un filtru MAF, s-a observat că valorile filtrului (M) influențează semnificativ nivelul de netezire aplicat semnalului afectat de zgomot. O valoare mai mare a lui M a condus la o mai bună reducere a zgomotului, însă, în același timp, a introdus și o întârziere mai mare în procesarea semnalului.

Observațiile obținute sugerează că alegerea corectă a parametrilor, cum ar fi pasul de timp și ordinea filtrului, este esențială pentru obținerea unor rezultate precise în procesarea semnalelor și în filtrarea zgomotului.